1. What is Postman?

Postman is a **tool for API testing**. You can use it to:

- Send HTTP requests (`GET`, `POST`, `PUT`, `PATCH`, `DELETE`, `OPTIONS`, `HEAD`)
- Validate responses (status codes, data, headers)
- Automate tests with scripts
- Manage environment variables (like tokens, base URLs)
- Run collections of requests (for regression or batch testing)

Think of it like a **browser for APIs**, but much more powerful.

---

2. Postman Interface Overview

When you open Postman, you'll see:

1. **Workspace** – Your working area.
2. **Collections** – Group of related requests (e.g., all User APIs in one collection).
3. **Requests** – Individual API calls (`GET /users`, `POST /users`, etc.).
4. **Environments** – Store variables like `baseUrl`, `token` for different environments (Dev/QA/Prod).
5. **Tabs under request URL**:

Tab	Purpose
Params	Query parameters like `?page=1&limit=10`
Authorization	Add API authentication (Bearer token, Basic Auth, OAuth)
Headers	Custom headers like `Content-Type: application/json`
Body	Request payload for POST/PUT/PATCH
Pre-request Script	Code that runs **before sending the request**
Tests	Code that runs **after receiving the response**
Cookies	View cookies sent by the server

Settings/More	Options for request

---

3. Types of HTTP Requests

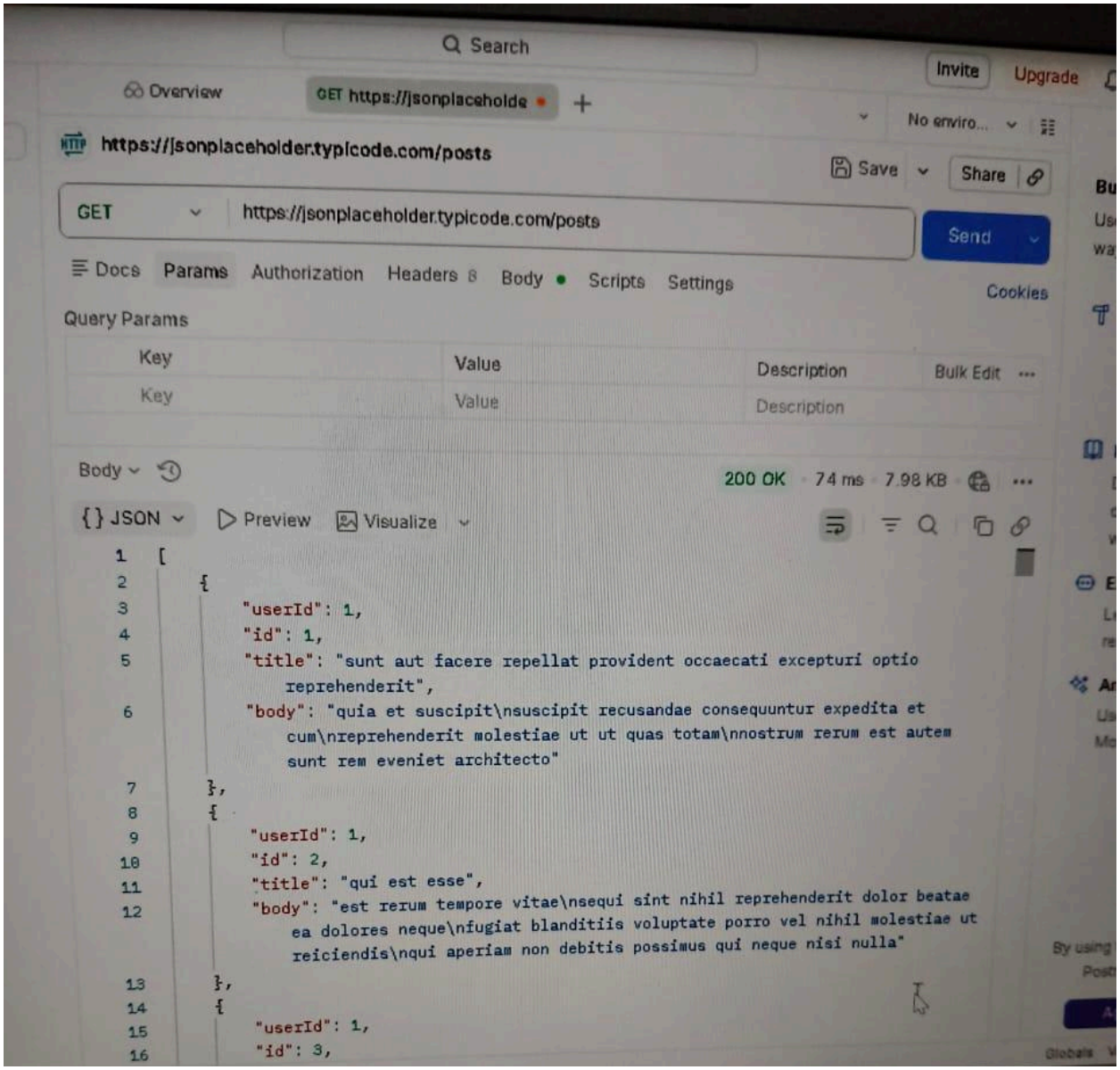
Method	Use Case
GET	Retrieve data (no body)
POST	Create new resource (send JSON body)
PUT	Update full resource
PATCH	Update partial resource
DELETE	Delete resource
HEAD	Get headers only
OPTIONS	Check allowed HTTP methods

---

4. How to Send Your First Request

Step 1: GET Request

1. Open Postman → Click **New** → **Request** → Name it `Get Users`.
2. Enter URL:
   https://jsonplaceholder.typicode.com/users
3. Select method: `GET`
4. Click **Send**
   → Response will appear below (JSON data)

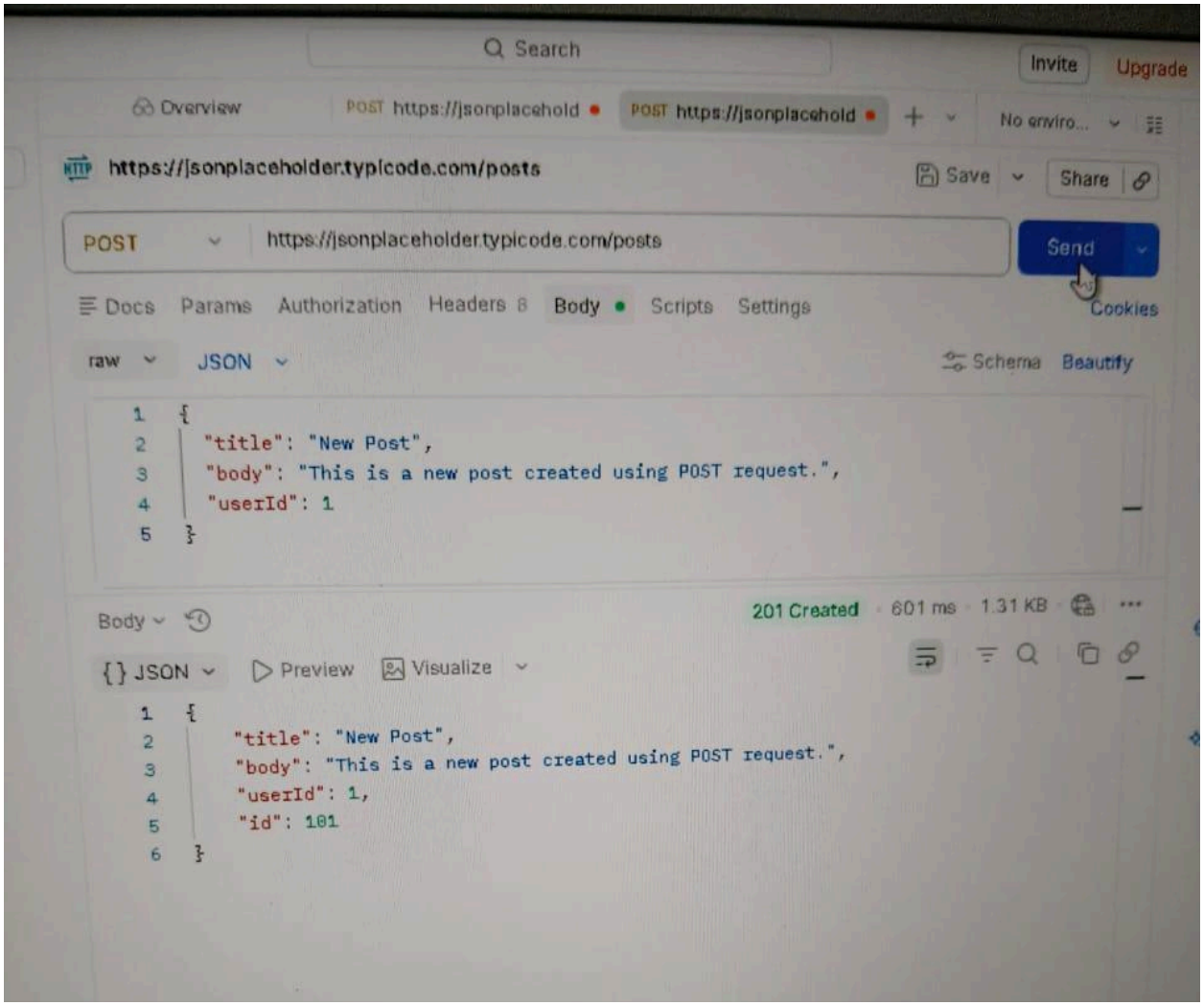


Step 2: POST Request

1. URL:
   https://jsonplaceholder.typicode.com/posts
2. Method: POST
3. Go to **Body** → **raw** → **JSON**:

```
{
 "title": "My Post",
 "body": "This is a test post",
 "userId": 1
}
```

4. Click **Send** → Response returns created resource (with `id`)



---

5. Using Headers

Some APIs require headers:

- `Content-Type: application/json`
- `Authorization: Bearer <token>`

How to add headers: Go to **Headers tab**, type key-value pairs.

---

6. Using Query Parameters

For GET requests:

- URL: `https://jsonplaceholder.typicode.com/posts`
- Params tab:
Key	Value
userId	1
- Result: API returns only posts with `userId = 1`

---

7. Environment Variables

- Click **Environments** → **Add**
- Example:
Variable	Value
baseUrl	https://jsonplaceholder.typicode.com
token	Bearer 123abc
- Use in requests:

{{baseUrl}}/posts

Benefit: Switch between Dev/QA/Prod easily.

---

8. Pre-request Script

- Runs **before the request**.
- Example: Generate token dynamically

```
pm.environment.set("token", "Bearer " + Math.random().toString(36).substring(2));
```

---

9. Tests Tab

- Runs **after the response**.
- Example: Validate status code and response field

```
pm.test("Status code is 200", () => {
  pm.response.to.have.status(200);
});
```

```
pm.test("Title partially updated", () => {
  const data = pm.response.json();
  pm.expect(data.title).to.eql("Partially Updated Title");
});
```

Where to find it:

- Open request → Tabs under URL → **Tests**
- Paste the code → Click **Send** → Results appear under **Test Results**

---

10. PATCH / PUT / DELETE Requests

PATCH Example

```
PATCH https://jsonplaceholder.typicode.com/posts/1
Body:
{
 "title": "Partially Updated Title"
}
```

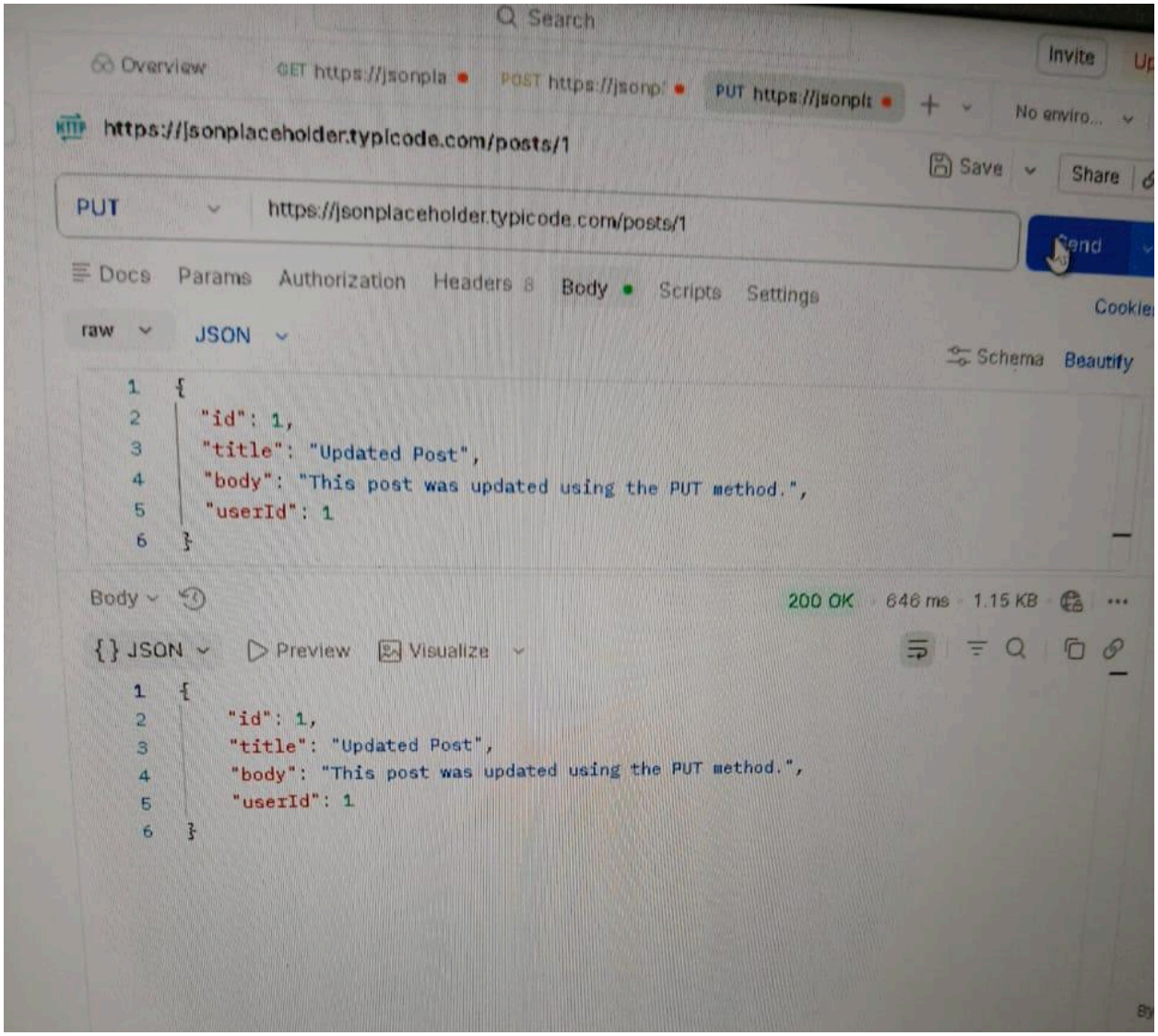
- Tests:

```
pm.test("Status code is 200", () => pm.response.to.have.status(200));
pm.test("Title updated", () => pm.expect(pm.response.json().title).to.eql("Partially Updated Title"));
```

PUT Example

- Updates entire resource:

```
PUT https://jsonplaceholder.typicode.com/posts/1
Body:
{
 "id": 1,
 "title": "Updated Title",
 "body": "Updated Body",
 "userId": 1
}
```

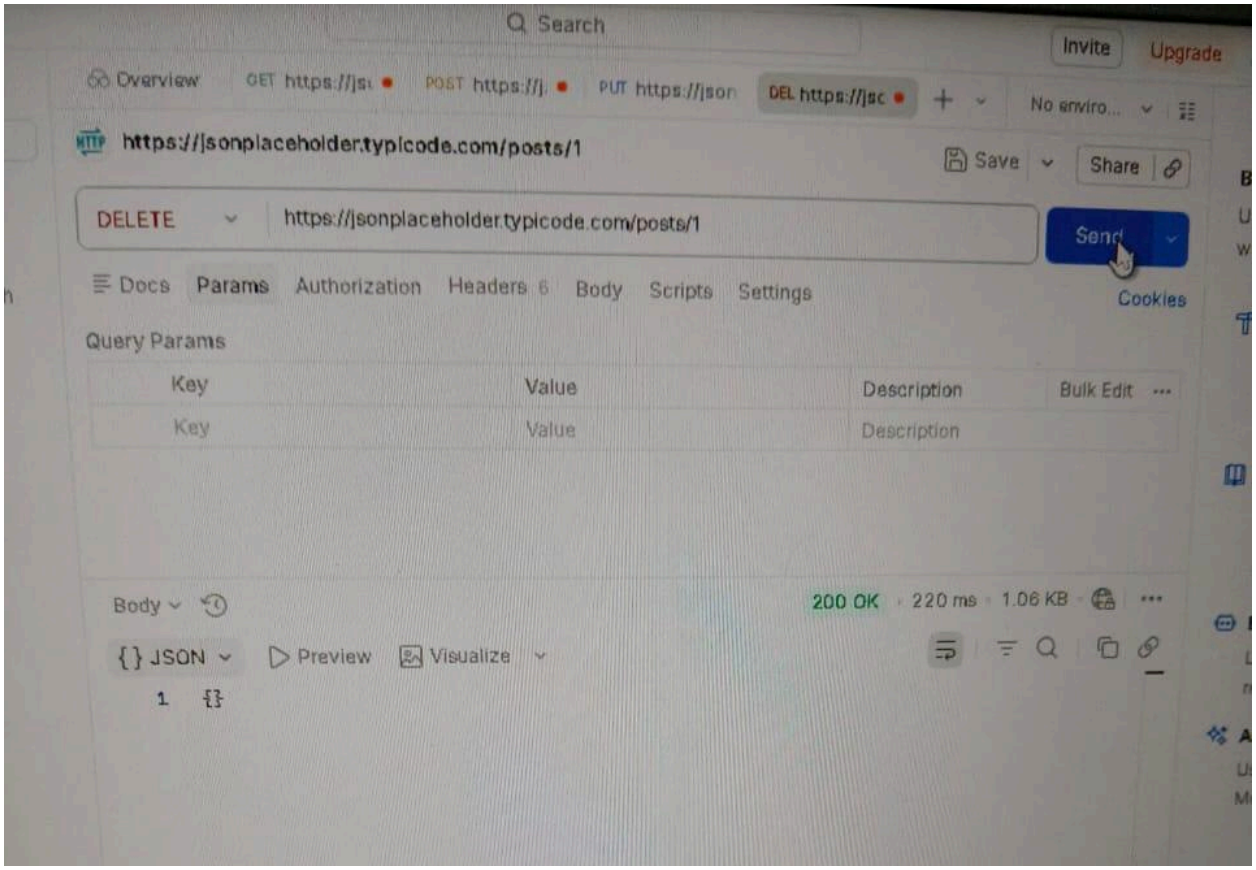


DELETE Example

- Delete resource:

DELETE https://jsonplaceholder.typicode.com/posts/1

- Status code usually `200` or `204`



---

11. Collection Runner

- Run multiple requests in sequence.

1. Click **Collections** → **Run**
2. Optionally use **CSV/JSON file** for data-driven testing.
3. Click **Start Run** → Postman executes all requests sequentially.

---

12. Monitors

- Scheduled runs for APIs.
- Postman sends requests at intervals → Notifies about failures → Useful for API uptime checks.

---

13. Assertions / Test Scripts

- Check status code, body, response time, headers

```
pm.test("Status 200", () => pm.response.to.have.status(200));
pm.test("Body contains userId", () =>
pm.expect(pm.response.json()).to.have.property("userId"));
pm.test("Response time < 2s", () =>
pm.expect(pm.response.responseTime).to.be.below(2000));
```

Summary Flow

1. **Create request** (GET/POST/PUT/PATCH/DELETE)
2. **Add headers / auth / query params / body**
3. **Set environment variables** if needed
4. **Add Pre-request script** for dynamic data
5. **Add Tests** for assertions
6. **Send request → Check Response**
7. **Add request to collection → Use Collection Runner**
8. **Schedule Monitors** if needed